

Wukong：迈向大规模推荐的扩展定律

Buyun Zhang^{*1} Liang Luo^{*1} Yuxin Chen^{*1}
Jade Nie¹ Xi Liu¹ Daifeng Guo¹ Yanli Zhao¹ Shen Li¹
Yuchen Hao¹ Yantao Yao¹ Guna Lakshminarayanan¹
Ellie Dingqiao Wen¹ Jongsoo Park¹
Maxim Naumov¹ Wenlin Chen¹

摘要

扩展定律对模型质量的可持续提升发挥着至关重要的作用。遗憾的是，由于现有推荐模型的扩展机制效率低下，它们尚未表现出类似于大语言模型领域所观察到的扩展定律。这一局限给这些模型适应日益复杂的真实世界数据集带来了重大挑战。本文提出了一种完全基于堆叠因子分解机的高效网络架构，以及一种协同扩展策略，统称为 Wukong，以在推荐领域建立扩展定律。Wukong 的独特设计使其只需增加层的高度和宽度，就能捕获多样化的任意阶交互。我们在六个公开数据集上进行了广泛评估，结果表明，Wukong 在模型质量方面始终优于 state-of-the-art 模型。此外，我们还在一个内部大规模数据集上评估了 Wukong 的可扩展性。结果显示，Wukong 在质量上保持了相对于 state-of-the-art 模型的优势，同时在跨越两个数量级的模型复杂度范围内维持扩展定律，并扩展至 100 GFLOP/example 以上，而以往方法在此范围内表现不足。

1. 引言

如今，基于深度学习的推荐系统 (DLRS) 为广泛的在线服务提供支持 (Naumov et al., 2019; Wang et al., 2021a; Lian et al., 2021; Liu et al., 2022; Covington et al., 2016)。

现代 DLRS 旨在处理连续稠密特征（如日期）与类别稀疏特征（如用户点击帖子历史）的混合输入。每个稀疏特征都通过可训练的嵌入查找表转换为稠密嵌入表示。随后，这些稠密嵌入被送入一个交互组件，以捕获特征之间复杂的交互。

尽管现有模型在较小数据集上展现出良好的准确率，

^{*} 同等贡献 ¹Meta AI. 通讯作者: Buyun Zhang <buyunz@meta.com>, Liang Luo <liangluo@meta.com>, Yuxin Chen <yuxinc@meta.com>。

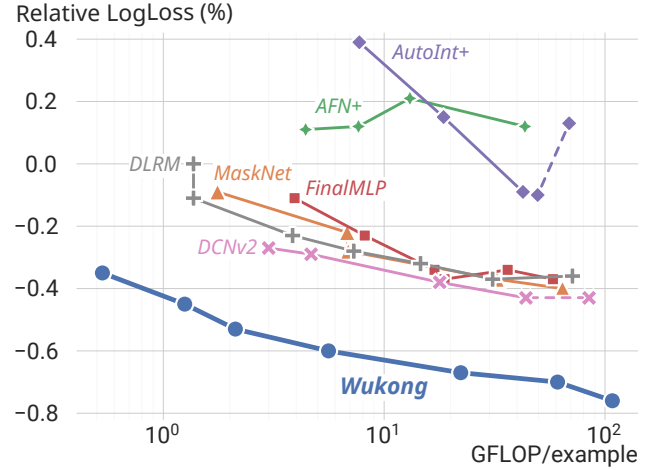


Figure 1: Wukong 在模型复杂度跨越两个数量级并超过 100 GFLOP/example 时，仍优于现有 state-of-the-art 模型，并在推荐领域展现出缩放定律。

但它们适应规模和复杂性都显著增大的数据集的能力，以及随模型规模扩大而持续提升质量的能力，仍不够确定。随着现代数据集呈指数级增长，这种可扩展性日益重要。例如，如今的生产数据集可能包含数千亿个训练样本 (Wang et al., 2021a)。此外，基础模型 (Bommasani et al., 2021) 也需要进行规模化扩展，以同时处理更大且更复杂的多种输入源。因此，能够依据不同的数据集规模和计算约束有效向上和向下扩展的 DLRS 至关重要。这种可扩展性体现在所谓的“缩放定律”中 (Kaplan et al., 2020)。

迄今为止，DLRS 扩展的主要趋势是通过稀疏扩展实现，即扩大嵌入表的规模（增加行数和/或提高维度），从而减少碰撞并增强表达能力。因此，DLRS 的参数量已达到万亿级 (Kang et al., 2020; Mudigere et al., 2021; Lian et al., 2021)，其中嵌入表占据了绝大部分参数。不幸的是，这种传统扩展方式存在若干实际缺点。仅仅扩大模型的稀疏组件并不能增强其捕获日益增多的特征之间复杂交互的能力。此外，这一趋势明显偏离硬件发展的趋势，因为下一代加速器的大多数改进集中在计算能力上 (Luo et al., 2018; 2017)，而嵌入表查找无法利用这些能力。因此，简单扩展嵌入表会导致高昂

的基础设施成本和较低的加速器利用率，尤其是在分布式环境中 (Luo et al., 2024)。

我们的工作旨在为推荐模型寻找一种替代性扩展机制，使其能够建立类似于 LLM 领域的扩展定律。具体而言，我们希望设计一种统一架构，使其质量能够通过协同策略，随着数据集规模、计算预算和参数预算的增长而持续提升。

我们专注于扩展交互组件，称为稠密扩展，以缓解稀疏扩展在质量和效率方面的缺陷。然而，由于种种原因，现有模型无法从这一范式中获益。例如，DLRM 缺乏捕获 < 高阶 > 交互的能力；DCNv2 和 AutoInt+ 缺乏有效的扩展策略，导致扩展时收益迅速递减；此外，即使采用残差连接 (He et al., 2016)、层归一化 (Ba et al., 2016)、梯度裁剪 (Pascanu et al., 2013) 等现代技巧，扩展现有模型仍容易出现训练稳定性问题 (Tang et al., 2023)。

为建立推荐模型的扩展定律，我们提出了 Wukong，这是一种简单的交互架构，展现出有效的稠密扩展特性。受二进制幂运算原理启发，我们的关键创新是使用一系列堆叠的因子分解机 (FM)，高效且可扩展地捕获任意阶特征交互。在我们的设计中，每个 FM 负责捕获相对于其输入的二阶交互，随后这些 FM 的输出由 MLP 转换为新的嵌入；这些嵌入对交互结果进行编码，并作为下一层的输入。

我们使用六个公开数据集和一个大规模内部数据集评估了 Wukong 的性能。结果表明，Wukong 在所有公开数据集上的 AUC 均优于 state-of-the-art 模型，这说明了 Wukong 架构的有效性，以及它在广泛的推荐任务和数据集上的泛化能力。在内部数据集上，Wukong 不仅在相当复杂度水平下的质量显著优于现有模型，而且当模型复杂度跨越两个数量级进行扩展并超过 100 GFLOP/example 时，其质量仍持续提升，而先前方法在此处已显不足。

2. 相关工作

深度学习推荐系统 (DLRS) 现有 DLRS 具有相似的结构。典型模型由稀疏组件和稠密组件组成。稀疏组件本质上是嵌入查找表，将稀疏类别特征转换为稠密嵌入；稠密组件则负责捕获这些嵌入之间的交互以生成预测。

稠密交互架构 捕获特征之间的交互是 DLRS 有效性的关键，下面重点介绍一些先前方法。AFN+ (Cheng et al., 2020) 将特征转换到对数空间以捕获任意阶交互；AutoInt+ (Song et al., 2019) 使用多头自注意力；DLRM 和 DeepFM (Naumov et al., 2019; Guo et al., 2017) 利用 Factorization Machines (FM) (Rendle, 2010) 显式捕获二阶交互；HOFM (Blondel et al., 2016) 对 FM 进行优化，以高效捕获更高阶交互；DCNv2 (Wang et al., 2021a) 使用 CrossNet，通过堆叠式特征交叉捕获交互，这可视为一种逐元素输入注意力形式。FinalMLP (Mao et al., 2023) 使用双线性融合来聚合两个 MLP 流的结果，每个流

均以特定于该流的门控特征作为输入。MaskNet (Wang et al., 2021b) 采用一系列 MaskBlocks 来捕获交互，将“输入注意力”应用于输入本身以及 DNN 的中间激活；xDeepFM (Lian et al., 2018) 将 DNN 与 Compressed Interaction Network 相结合，后者通过外积捕获交互，并通过逐元素求和和压缩结果。

扩大 DLRS 规模 (Kang et al., 2020; Mudigere et al., 2021; Lian et al., 2021) 提供了稀疏扩展机制。(Shin et al., 2023) 聚焦于扩大用户表示模型的规模，其中报告的最大模型总计算量小于 0.1 PF-days；(Zhang et al., 2023) 旨在改进用户侧的序列建模，其中报告的最大模型参数量小于 0.8B。此外，(Ardalani et al., 2022) 研究了 DLRM 的缩放定律；我们将其作为基线，并在实验中进一步扩大其规模。与此正交的是，(Zhao et al., 2023b) 提出一种以用户为中心的排序建模方式来提升可扩展性；(Guo et al., 2023) 提供了有关稀疏扩展的洞见，展示了已有方法的局限，是对我们工作的补充。此外，VIP5 (Geng et al., 2023) 利用 LLM 中现有的缩放定律，将多模态 LLM 应用于推荐；然而，(Lin et al., 2023) 指出，仍需进一步研究以验证在基于 LLM 的推荐器中规模更大是否意味着性能更好，而 (Huang et al., 2024) 则认为，需要在更多样化的数据集上进行评估才能得出结论。

3. Wukong 的设计

在设计 Wukong 架构时，我们始终考虑两个目标：(1) 有效捕获错综复杂的高阶特征交互；(2) 确保 Wukong 的质量能够随数据集大小、GFLOP/example 和参数预算平稳扩展。

3.1. 概述

在 Wukong 中，类别特征和稠密特征首先通过一个 **嵌入层** (第 3.2 节)，该层将这些输入转换为稠密嵌入。

如图 2 所示，Wukong 随后采用一个 **交互堆栈** (第 3.3 节)，即由统一神经网络层组成的堆栈，用于捕获嵌入之间的交互。交互堆栈的灵感来自二进制幂的概念，使每个后续层都能捕获阶数呈指数增长的交互。交互堆栈中的每一层都由一个 **因子分解机块** (FMB, 第 3.4 节) 和一个 **线性压缩块** (LCB, 第 3.5 节) 组成。FMB 和 LCB 分别接收上一层的输入，并将两者的输出集成作为当前层的输出。交互堆栈之后是最终的多层感知机 (MLP) 层，它将交互结果映射为预测。

3.2. 嵌入层

给定一个多热类别输入，嵌入表会将其映射为稠密嵌入。这一过程包含一系列查找操作，每次查找对应输入中的一个“热”维度。随后使用池化操作（通常为求和）聚合查找结果。

在我们的设计中，嵌入层生成的所有嵌入均采用统一的嵌入维度，称为全局嵌入维度 d 。为适应不同特征重

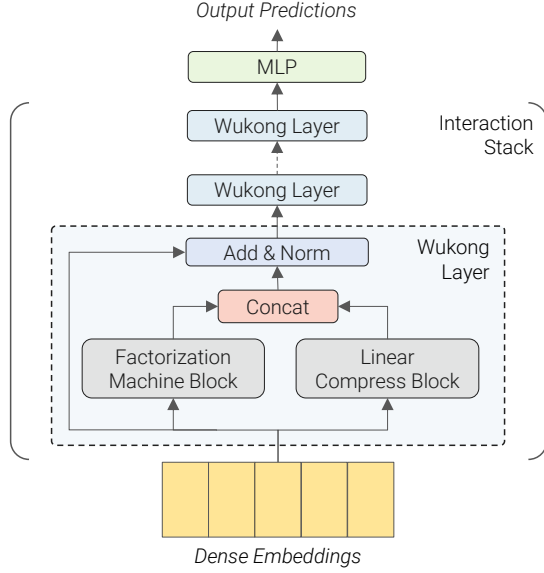


Figure 2: Wukong 使用交互堆栈来捕获特征交互。堆栈中的每一层都由一个 Factorization Machine Block 和一个 Linear Compress Block 组成。

要性的差异，每个被视为重要的特征会生成多个嵌入。较不重要的特征则分配较小的底层嵌入维度。随后，将这些较小的嵌入集中分组、拼接，并使用 MLP 转换为 d 维嵌入。

稠密输入经 MLP 转换为具有相同 d 维度的潜在嵌入，并与类别输入的嵌入输出合并。由此得到大小为 $X_0 \in \mathbb{R}^{n \times d}$ 的输出张量，其中 n 是稠密部分与稀疏部分的嵌入总数。随后， X_0 即可由交互堆栈进一步处理。

请注意，与 DCN (Wang et al., 2021a) 等传统方法不同，我们将每个嵌入向量视为一个完整单元（后文详述），因此将其表示为 $X_0 \in \mathbb{R}^{n \times d}$ ，而非 $X_0 \in \mathbb{R}^{nd}$ 。

3.3. 交互堆栈

交互模块堆叠了 l 个相同的交互层，其中每一层都使用因子分解机 (FM) 逐步捕获更高阶的特征交互。

一个交互层包含两个并行的模块：因子分解机模块 (FMB) 和线性压缩模块 (LCB)。FMB 计算该层输入嵌入之间的特征交互，而 LCB 仅将该层线性压缩后的输入嵌入向前传递。随后将 FMB 和 LCB 的输出拼接起来。

对于堆栈中的第 i 层，其结果可以包含从 1 阶到 2^i 阶的任意阶特征交互。这一点可通过归纳法简单证明。假设第 i 层的输入包含从 1 阶到 2^{i-1} 阶的交互；对于第一层（即 $i = 1$ ），该假设成立。由于 FMB 会生成 $(o_1 + o_2)$ 阶特征交互，而其输入是 o_1 阶和 o_2 阶交互，因此立即可得第 i 层的输出包含从 1 阶到 2^i 阶的交互；其下界来自 LCB 的输出，上界则由 FM 对输入中的两个 2^{i-1} 阶交互进行交互而达到。

为帮助稳定训练，我们还采用了跨层残差连接，随后进行层归一化 (LN)。综合以上内容，可得

$$X_{i+1} = \text{LN}(\text{concat}(\text{FMB}_i(X_i), \text{LCB}_i(X_i)) + X_i)$$

根据 FMB 和 LCB 的具体配置， X_i 的嵌入数量可能与 X_{i+1} 不同，这种情况通常发生在第一层。为处理这种情况，可对残差进行线性压缩以匹配形状。

3.4. 因子分解机模块 (FMB)

一个 FMB 包含一个 FM，后接一个 MLP。FM 用于捕获输入嵌入的显式特征交互，其输出为一个二维交互矩阵，其中每个元素表示一对嵌入之间的交互。该交互矩阵经过展平并通过 MLP 转换为形状为 $(n_F \times d)$ 的向量，再重塑为 n_F 个嵌入供后续使用。

在操作层面，FMB 执行以下步骤：

$$\text{FMB}(X_i) = \text{reshape}(\text{MLP}(\text{LN}(\text{flatten}(\text{FM}(X_i))))$$

Wukong 的 FM 模块可以完全自定义：例如，在最基础的版本中，我们遵循 (Naumov et al., 2019) 中的 FM 设计，即计算所有嵌入向量对之间的点积 $\text{FM}(X) = XX^T$ 。我们在第 3.6 节讨论了更优化的 FM 设计。

3.5. 线性压缩模块 (LCB)

LCB 仅对嵌入进行线性重组而不增加交互阶数，这对于确保交互阶数的不变性贯穿各层至关重要。具体而言，它保证第 i 个交互层捕获从 1 到 2^i 阶的交互。LCB 执行的操作可描述如下：

$$\text{LCB}(X_i) = W_L X_i$$

其中， $W_L \in \mathbb{R}^{n_L \times n_i}$ 是权重矩阵， n_L 是表示压缩嵌入数量的超参数， n_i 是第 i 层的输入嵌入数量。

3.6. 优化的 FM

由于采用成对点积，FM 的计算与存储复杂度随嵌入数量呈二次增长；在包含数千个特征的真实世界数据集上，这会迅速变得难以承受。

为在降低计算成本的同时实现有效的特征交互，我们采用了与 (Sharma, 2023; Anonymous, 2019) 类似的方案，利用成对点积矩阵的低秩性质；这一性质已在许多真实世界数据集中观察到 (Wang et al., 2021a)。

当 $d \leq n$ 时，点积交互 XX^T 是一个秩为 d 的矩阵；当大型数据集的特征数量大于嵌入维度时，通常就是这种情况。因此，在理论上不损失信息的情况下，我们可以有效地将输出矩阵的大小从 $n \times n$ 缩减到 $n \times k$ ，其中 k 是超参数；具体做法是将 XX^T 与可学习投影矩阵 Y 相乘，该矩阵的形状为 $n \times k$ （即计算 $XX^T Y$ ）。这降低了存储交互矩阵所需的内存。随后，我们可以利

用结合律先计算 $X^T Y$ ，进一步将计算复杂度从 $O(n^2 d)$ 降至 $O(nkd)$ ，其中 $k \ll n$ 。

此外，为提升模型质量，可通过使用 MLP 处理线性压缩后的输入，使投影矩阵 Y 能够感知输入。除非另有说明，后续实验默认使用优化后的 FM。

3.7. 复杂度分析

我们假设交互堆栈中的每一层都使用相同的超参数，并且 MLP 中最大的 FC 大小为 h 。

对于第一层，FMB 的时间复杂度是 FM 与 MLP 的时间复杂度之和，二者分别为 $O(nkd) \approx O(ndh)$ 和 $O(nkh + h^2 + n_F dh) \approx O(ndh + h^2)$ 。LCB 的时间复杂度为 $O(nn_L d) \approx O(ndh)$ 。对于后续各层，其时间复杂度为 $O(n' dh + h^2)$ ，其中 $n' = n_L + n_F$ 。因此，Wukong 的总时间复杂度为 $O(ndh + \ln' dh + h^2) \approx O(ndh \log n + h^2)$ 。

3.8. 扩展 Wukong

下面我们汇总与规模扩展相关的主要超参数，随后介绍我们围绕这些超参数扩大 Wukong 规模所做的工作。

- l : 交互堆栈中的层数。
- n_F : FMB 生成的嵌入数量
- n_L : LCB 生成的嵌入数量
- k : 优化 FM 中压缩嵌入的数量
- MLP: FMB 的 MLP 中的层数与 FC 大小

在扩大规模时，我们首先着重增加 l ，使模型能够捕获更高阶的交互。随后，我们增大其他超参数，以增强模型捕获更广泛交互的能力。

3.9. Wukong 有效性提升背后的直觉

与以 FM 作为主要交互架构的现有工作相比，Wukong 以创新方式堆叠 FM，极大增强了传统 FM 的能力。这使 Wukong 能够捕获任意阶的交互，因此对于需要高阶推理的大规模复杂数据集尤为有效。尽管已有一些高阶 FM 方面的探索，但 Wukong 以指数速率捕获高阶交互，具有很高的效率，既绕过了 HOFM 中的线性复杂度，也避免了 xDeepInt 中代价高昂的外积。

尽管 MLP 在隐式捕获交互方面表现出局限性 (Beutel et al., 2018)，Wukong 不同于依赖 MLP 捕获交互的方法。相反，Wukong 主要使用 MLP 将交互结果转换为嵌入表征，随后将其用于进一步交互。MLP 的这种独特用法增强了模型有效处理和解释复杂异构特征的能力。

此外，Wukong 将每个嵌入视为一个整体，专注于嵌入级交互。与捕获逐元素交互的架构相比，这种方法显著降低了计算需求。

	# 样本	# 特征
Frappe	0.29M	10
MicroVideo	1.7M	7
MovieLens Latest	2M	3
KuaiVideo	13M	8
TaobaoAds	26M	21
Criteo Terabyte	4B	39
内部数据集	146B	720

Table 1: 评估数据集的统计信息。

4. 实现

本节讨论在大规模数据集上有效训练高复杂度 Wukong 的实践。

总体而言，要使 Wukong 的训练切实可行，需要采用分布式训练。对于嵌入层，我们使用 Neo (Mudigere et al., 2021) 和 NeuroShard (Zha et al., 2023) 提供的按列分片嵌入包实现。对于稠密部分，我们采用 FSDP (Zhao et al., 2023a) 并调节分片因子，在性能与内存容量之间取得平衡，使模型能够装入内存，同时避免产生过多冗余。

为提高训练效率，我们采用自动算子融合来提升训练性能。此外，我们积极应用量化，以同时降低计算、内存和通信开销。具体而言，我们以 FP16 训练 Wukong 的嵌入表，并在前向传播中以 FP16、在反向传播中以 BF16 传输嵌入查找结果；在反向传播中传输稠密参数的梯度时，我们使用 BF16 量化。

5. 评估概览

我们使用六个公开数据集和一个内部数据集评估 Wukong，其详细信息汇总于表 1。这些评估的结果分为两节呈现。

在第 6 节中，我们在六个公开数据集上进行评估，重点展示 Wukong 在低复杂度范围内的有效性。结果表明，Wukong 在全部六个数据集上均超越以往的 state-of-the-art 方法，证明了其有效性。

在第 7 节中，我们在大规模内部数据集上进行评估，以展示 Wukong 的可扩展性。与规模最大的一个数据集 Criteo 相比，该数据集的样本数多 30 倍、特征数多 20 倍。结果显示，(1) 在模型质量和运行速度两方面，Wukong 均持续优于所有基线模型，并且在所有复杂度尺度上都保持这一优势；(2) 与基线模型相比，Wukong 呈现出更优的扩展趋势。我们还进行了消融研究，以了解 Wukong 中各个组件的独立贡献及其有效性。

6. 公开数据集上的评估

本节旨在展示 Wukong 在多种公开数据集上的有效性。除非另有说明，为与先前工作保持一致，我们使用 BARS 基准 (Zhu et al., 2022b) 提供的预处理流程。

	Frappe		MicroVideo		MovieLens L.		KuaiVideo		TaobaoAds		Criteo TB	
	AUC	LogLoss	AUC	LogLoss	AUC	LogLoss	AUC	LogLoss	AUC	LogLoss	AUC	LogLoss
基线												
AFN+	0.9812	0.2340	0.7220	<u>0.4142</u>	0.9648	0.3109	0.7348	0.4372	0.6416	0.1929	0.8023	0.1242
AutoInt+	0.9806	0.1754	0.7155	0.4203	0.9693	0.2178	0.7297	0.4376	0.6437	0.1930	0.8073	0.1233
DCNv2	0.9774	0.2325	0.7187	0.4162	0.9683	0.2169	0.7360	0.4383	0.6457	<u>0.1926</u>	0.8096	0.1227
DLRM	0.9846	0.1465	0.7173	0.4179	0.9685	0.2160	0.7357	0.4382	0.6430	0.1931	0.8076	0.1232
FinalMLP	0.9868	<u>0.1280</u>	0.7247	0.4147	0.9723	0.2211	0.7374	0.4435	0.6434	0.1928	0.8096	0.1226
MaskNet	0.9816	0.1701	0.7255	0.4157	0.9676	0.2383	0.7376	0.4372	0.6433	0.1927	0.8100	0.1227
xDeepFM	0.9780	0.2441	0.7167	0.4172	0.9667	0.2089	0.7118	0.4565	0.6342	0.1961	0.8084	0.1229
本文方法												
Wukong	0.9868	0.1757	0.7292	0.4148	0.9723	<u>0.1794</u>	0.7414	<u>0.4367</u>	0.6488	0.1954	0.8106	<u>0.1225</u>

Table 2: 六个公开数据集上的评估结果。每个数据集上具有最佳 AUC 和最佳 LogLoss 的模型以突出格式标出。

6.1. 通用评估设置

6.1.1. 数据集

Frappe (Baltrunas) 是一份应用使用日志。该数据集用于预测用户是否会在给定上下文中使用某个应用。

MicroVideo (Chen et al., 2018) 是 THACIL 工作提供的、基于内容理解的数据集，包含用户与微视频之间的交互。该日志包含多模态嵌入，以及传统特征。

MovieLens Latest (Harper & Konstan, 2015) 是一个广为人知的数据集，包含用户对电影的评分。

KuaiVideo (Kuaishou) 是快手发布的竞赛数据集，用于预测用户点击新微视频的概率。除其他类别特征和浮点特征外，该数据集也包含基于内容理解的嵌入。

TaobaoAds (Tianchi, 2018) 该数据集包含淘宝上连续 8 天的广告点击率 (CTR) 预测数据。

Criteo Terabyte (Criteo) 该数据集包含连续 24 天的广告点击反馈。我们使用最后一天的数据进行测试。

6.1.2. 基线

我们将 Wukong 与学术界和工业界广泛采用的七种知名 state-of-the-art 模型进行基准比较，包括 AFN+ (Cheng et al., 2020)、AutoInt+ (Song et al., 2019)、DLRM (Naumov et al., 2019)、DCNv2 (Wang et al., 2021a)、FinalMLP (Mao et al., 2023)、MaskNet (Wang et al., 2021b) 和 xDeepFM (Lian et al., 2018)。

6.1.3. 指标

AUC 曲线下面积 (AUC) 衡量模型在所有阈值下正确区分正样本与负样本的能力，数值越高越好。遵循推荐领域惯例 (Tien et al., 2014; Blondel et al., 2016; Song et al., 2019; Wang et al., 2021a; Zhu et al., 2022b; Mao et al., 2023)，我们以 AUC 作为超参数调优的依据和报告中的首要指标。

LogLoss 对数损失根据预测值与真实标签之间的偏差程度量化惩罚，数值越低越好。

6.2. 模型特定设置

对于除 Criteo 之外的五个较小数据集，我们采用公开的 BARS 评估框架 (Zhu et al., 2022a; 2021)。只要条件允许，我们就直接使用 BARS 搜索得到的最佳模型配置，其余模型则使用所提供的默认超参数。除框架提供的默认嵌入维度外，我们还测试了 128 的嵌入维度，并报告这两种配置中结果更优者。对于 Wukong，我们调节 dropout 率、优化器设置和 LCB 压缩程度，以适应特征数量。

我们利用更大的 Criteo 数据集评估模型在采用单遍训练的真实在线推荐系统中的性能。鉴于新的训练设置，我们使用第 4 节所述系统，对所有基线和 Wukong 开展了广泛的网格搜索，以便进行公平比较。这一穷举过程包含近 3000 次独立运行。附录 A 给出了针对各模型的搜索空间。随后，第 7 节以搜索得到的最佳模型超参数作为基础配置。

6.3. 结果

我们在表 2 中汇总了结果。总体而言，Wukong 能够在所有公开数据集上取得 AUC 指标的最先进结果。该结果证明了 Wukong 架构的有效性，以及其理解多样化数据集并泛化到广泛推荐任务的能力。

7. 内部数据集上的评估

在本节中，我们使用一个大规模数据集展示 Wukong 的可扩展性，并深入理解 Wukong 的不同组件如何各自促进其有效性；该数据集使我们能够研究在小型公开数据集中无法观察到的新兴特性。

7.1. 评估设置

7.1.1. 数据集

该数据集共包含 146B 条记录，并具有 720 个不同的特征。每个特征描述物品或用户的一项属性。该数据集包含两个相关任务：(Task1) 预测用户是否对某个物品表现出兴趣（例如点击），以及 (Task2) 是否发生转化（例如点赞、关注）。

7.1.2. 指标

GFLOP/example 每个样本的十亿次浮点运算 (GFLOP/example) 量化模型训练期间的计算复杂度。

PF-days 训练计算总量，等价于一台以 1 PetaFLOP/s 运行的机器持续运行 1 天。

#Params 以模型参数数量衡量的模型大小。稀疏嵌入表大小固定为 627B 个参数。

Relative LogLoss 相对于固定基线的 LogLoss 改善。我们选择采用基础配置的 DLRM 作为基线。在该数据集上，0.02% 的 Relative LogLoss 改善被视为显著。我们报告在线训练期间最后一个 1B 窗口上的相对 LogLoss。

7.1.3. 基线

我们遵循第 6.1.2 节中详述的相同基线设置。然而，报告结果中未包含 xDeepFM，这是因为其代价高昂的外积操作与大规模数据集不兼容，即使在最小配置下也会持续引发内存不足问题。

7.1.4. 训练

所有实验均采用我们在先导研究中找到的最佳优化器配置，即稠密部分使用 Adam ($\text{lr}=0.04$ 、 $\text{beta1}=0.9$ 、 $\text{beta2}=1$)，稀疏嵌入表使用 Rowwise Adagrad ($\text{lr}=0.04$)。模型以在线训练方式进行训练和评估。所有运行的嵌入维度均固定为 160。

我们以第 6 节所述 Criteo Terabyte 评估中找到的最佳配置作为超参数起点，并逐步增加每个模型的参数量。所有实验均使用 262,144 的全局批大小。每项实验根据模型大小在 128 或 256 块 H100 GPU 上运行。

7.2. 结果

我们在两个任务上观察到了相当的结果，并在正文中报告 Task1 的结果，而 Task2 的详细结果见附录 C。

质量与计算复杂度 图 1 展示了质量与计算复杂度（经验上为 $y = -100 + 99.56x^{0.00071}$ ）之间的关系。结果表明，Wukong 在各种复杂度水平上始终优于所有基线，LogLoss 提升超过 0.2%。值得注意的是，Wukong 在跨越两个数量级的模型复杂度范围内保持其缩放定律——大致相当于复杂度每增加四倍，提升 0.1%。在基线模型中，AFN+、DLRM 和 FinalMLP 往往在达到一定复

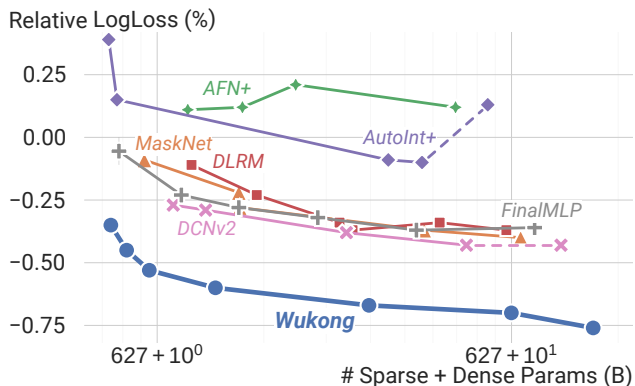


Figure 3: Wukong 在内部数据集上相对于 # 参数数量的可扩展性。

杂度后进入平台期，而 AutoInt+、DCNv2 和 MaskNet 未能进一步提升质量¹。尽管如此，即使是表现最佳的基线 DCNv2，也需要将复杂度提高 40 倍才能达到 Wukong 的质量。

质量与模型大小 图 3 展示了模型质量与模型大小之间的相关性。与上述计算复杂度扩展中观察到的趋势一致，Wukong 在所有模型大小尺度上均始终以约 0.2% 的优势超过所有基线，同时呈现稳定的提升趋势，直至超过 6370 亿个参数²。

质量与数据大小 参见附录 E。

模型特定的扩展 在整个扩展过程中，我们针对每个模型采用了不同策略。每次运行的详细超参数设置见附录 C。各模型的扩展过程总结如下：

Wukong 我们通过调节第 3.8 节详述的超参数来扩展 Wukong。

AFN+ 我们扩展了 AFN 的隐藏层、集成 DNN 和对数神经元数量。结果表明，扩展 AFN 并不能提升模型质量。

AutoInt+ 我们扩展了多头注意力和集成 DNN。该模型的初始模型质量比其他模型差，但在扩展后有明显改善。

DLRM 我们扩展了顶部 MLP。结果表明，复杂度超过 31 GFLOP/example 后，模型质量开始饱和。

DCNv2 我们同时扩展了 Cross Network 和 Deep Network。扩展 Cross Network 并未带来任何质量提升。DCNv2 的训练稳定性逊于其他模型，因此我们采用了严格的梯度裁剪。

¹AutoInt+ 和 DCNv2 在进一步扩展时始终面临严重的训练不稳定问题。AutoInt+ 从损失爆炸中恢复，但模型质量有所下降；而 DCNv2 未能恢复，其质量根据爆炸前的性能进行估算。MaskNet 受制于过高的内存消耗，导致内存不足错误，阻碍了进一步扩展。

²我们验证了 Wukong 在线上 and 离线设置中的有效性；为简洁起见，我们重点报告离线指标。

FinalMLP 我们扩展了两条 MLP 流和 Feature Selection 模块。结果表明，模型质量在低复杂度区域有所改善，但超过 36 GFLOP/example 后开始饱和。

MaskNet 我们测试了 Parallel 和 Serial MaskNet，并发现 Parallel 变体表现更好。我们降低了初始缩减比，以确保模型规模能够实际运行，并逐步扩展 MaskBlocks 的数量、DNN 和缩减比。

7.3. 消融实验

各组件的重要性 我们的目标是展示 FMB、LCB 和残差连接在 Wukong 交互堆栈中的重要性。为此，我们开展了逐个停用组件的实验，具体方法是将相应组件的结果置零。

如图 4 所示，将 FMB 的结果置零会导致模型质量大幅下降。有趣的是，单独停用 LCB 或残差连接只会造成较小的质量下降，而同时停用二者则会导致质量显著下降。这一观察表明，通过对 FMB 输出进行零填充并引入残差连接，可以简化 LCB。

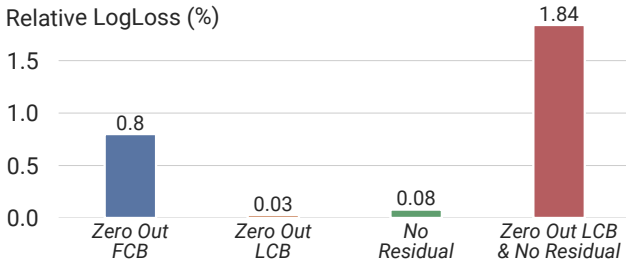


Figure 4: 各组件的重要性。

扩大单个组件规模的影响 我们旨在剖析扩大 Wukong 中各个超参数时，它们对模型质量的贡献。我们从一个基础配置出发，随后逐步将每个超参数增大一倍。结果如图 5 所示。我们观察到，增加 Wukong 的层数 l 会显著提升模型质量，这是因为模型能够捕获更高阶的交互。此外，增大 MLP 的规模也会带来可观的性能提升。增大 k 和 n_F 被证明是有益的，而在基础配置下， n_L 的收益已趋于饱和。值得注意的是，联合扩大 k, n_F, n_L 的规模所带来的质量提升，比逐个扩大它们更为显著。

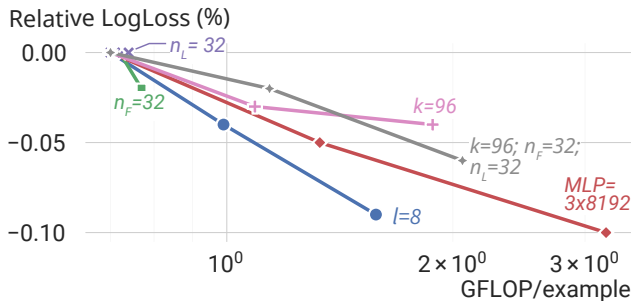


Figure 5: 扩大单个组件规模的影响。

8. 讨论

扩展模型的实际服务 扩展到高复杂度会给实时服务带来显著挑战。潜在解决方案包括训练一个多任务基础模型以摊销成本：将大模型的知识蒸馏到小型、高效的模型中用于服务。

局限性与未来工作 我们也注意到本工作的局限与注意事项，它们可作为未来工作的目标。

理解 Wukong 可扩展性的确切极限是一个重要研究方向。由于计算需求巨大，我们尚未达到这一极限开始显现的复杂度水平。

尽管 Wukong 在多种评估中表现出卓越质量，但对其基本原理的全面理论理解，尤其是与同样采用堆叠点积结构的 Transformer 等架构相比，仍有待进一步探索。

此外，Wukong 在推荐之外的泛化能力，特别是在输入数据源具有异构性、类似于推荐系统中不同特征的领域，仍需进一步探索和理解。

9. 结论

我们提出了一种名为 Wukong 的有效网络架构。我们证明了 Wukong 在推荐领域建立了此前未被观察到的缩放定律——Wukong 能够在两个数量级的计算复杂度范围内高效地扩展和缩减，同时保持相对于其他最先进模型的竞争优势。这使其成为一种可扩展架构，能够作为骨干网络服务于广泛任务和数据集上从小型垂直模型到大型基础模型的各种模型。

影响声明

本文所介绍工作的目标是推动机器学习领域的发展。我们的工作可能产生许多社会影响，但我们认为其中没有需要在此特别强调的影响。

References

- Anonymous. Dot product matrix compression for machine learning. *Technical Disclosure Commons*, 2019.
- Ardalani, N., Wu, C.-J., Chen, Z., Bhushanam, B., and Aziz, A. Understanding scaling laws for recommendation models. *arXiv preprint arXiv:2208.08489*, 2022.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Baltrunas, L. Frappe - mobile app usage. URL <https://www.baltrunas.info/context-aware>.
- Beutel, A., Covington, P., Jain, S., Xu, C., Li, J., Gatto, V., and Chi, E. H. Latent cross: Making use of context in recurrent recommender systems. In *Proceedings of the eleventh ACM international conference on web search and data mining*, pp. 46–54, 2018.

- Blondel, M., Fujino, A., Ueda, N., and Ishihata, M. Higher-order factorization machines. *Advances in Neural Information Processing Systems*, 29, 2016.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosse-lut, A., Brunskill, E., et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- Chen, X., Liu, D., Zha, Z.-J., Zhou, W., Xiong, Z., and Li, Y. Temporal hierarchical attention at category- and item-level for micro-video click-through prediction. In *MM*, 2018.
- Cheng, W., Shen, Y., and Huang, L. Adaptive factorization network: Learning adaptive-order feature interactions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 3609–3616, 2020.
- Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*, pp. 191–198, 2016.
- Criteo. Criteo 1tb click logs dataset. <https://ailab.criteo.com/download-criteo-1tb-click-logs-dataset/>.
- Geng, S., Tan, J., Liu, S., Fu, Z., and Zhang, Y. Vip5: Towards multimodal foundation models for recommendation. *arXiv preprint arXiv:2305.14302*, 2023.
- Gui, H., Wang, R., Yin, K., Jin, L., Kula, M., Xu, T., Hong, L., and Chi, E. H. Hiformer: Heterogeneous feature interactions learning with transformers for recommender systems. *arXiv preprint arXiv:2311.05884*, 2023.
- Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: a factorization-machine based neural network for ctr prediction. *arXiv preprint arXiv:1703.04247*, 2017.
- Guo, X., Pan, J., Wang, X., Chen, B., Jiang, J., and Long, M. On the embedding collapse when scaling up recommendation models. *arXiv preprint arXiv:2310.04400*, 2023.
- Harper, F. M. and Konstan, J. A. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5 (4), dec 2015. ISSN 2160-6455. doi: 10.1145/2827872. URL <https://doi.org/10.1145/2827872>.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- Huang, C., Yu, T., Xie, K., Zhang, S., Yao, L., and McAuley, J. Foundation models for recommender systems: A survey and new perspectives. *arXiv preprint arXiv:2402.11143*, 2024.
- Kang, W.-C., Cheng, D. Z., Yao, T., Yi, X., Chen, T., Hong, L., and Chi, E. H. Learning to embed categorical features without embedding tables for recommendation. *arXiv preprint arXiv:2010.10784*, 2020.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Kuaishou. URL <https://www.kuaishou.com/activity/uimc>.
- Lian, J., Zhou, X., Zhang, F., Chen, Z., Xie, X., and Sun, G. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 1754–1763, 2018.
- Lian, X., Yuan, B., Zhu, X., Wang, Y., He, Y., Wu, H., Sun, L., Lyu, H., Liu, C., Dong, X., Liao, Y., Luo, M., Zhang, C., Xie, J., Li, H., Chen, L., Huang, R., Lin, J., Shu, C., Qiu, X., Liu, Z., Kong, D., Yuan, L., Yu, H., Yang, S., Zhang, C., and Liu, J. Persia: An open, hybrid system scaling deep learning-based recommenders up to 100 trillion parameters. November 2021.
- Lin, J., Dai, X., Xi, Y., Liu, W., Chen, B., Li, X., Zhu, C., Guo, H., Yu, Y., Tang, R., et al. How can recommender systems benefit from large language models: A survey. *arXiv preprint arXiv:2306.05817*, 2023.
- Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., et al. Monolith: Real time recommendation system with collisionless embedding table. *corr abs/2209.07663* (2022), 2022.
- Luo, L., Liu, M., Nelson, J., Ceze, L., Phanishayee, A., and Krishnamurthy, A. Motivating in-network aggregation for distributed deep neural network training. In *Workshop on Approximate Computing Across the Stack*, 2017.
- Luo, L., Nelson, J., Ceze, L., Phanishayee, A., and Krishnamurthy, A. Parameter hub: a rack-scale parameter server for distributed deep neural network training. In *Proceedings of the ACM Symposium on Cloud Computing*, pp. 41–54, 2018.
- Luo, L., Zhang, B., Tsang, M., Ma, Y., Chu, C.-H., Chen, Y., Li, S., Hao, Y., Zhao, Y., Lakshminarayanan, G., et al. Disaggregated multi-tower: Topology-aware modeling technique for efficient large-scale recommendation. *arXiv preprint arXiv:2403.00877*, 2024.

- Mao, K., Zhu, J., Su, L., Cai, G., Li, Y., and Dong, Z. Finalmlp: An enhanced two-stream mlp model for ctr prediction. *arXiv preprint arXiv:2304.00902*, 2023.
- Mudigere, D., Hao, Y., Huang, J., Tulloch, A., Sridharan, S., Liu, X., Ozdal, M., Nie, J., Park, J., Luo, L., et al. High-performance, distributed training of large-scale deep learning recommendation models. *arXiv preprint arXiv:2104.05158*, 2021.
- Naumov, M., Mudigere, D., Shi, H.-J. M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C.-J., Azzolini, A. G., et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091*, 2019.
- Pascanu, R., Mikolov, T., and Bengio, Y. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pp. 1310–1318. Pmlr, 2013.
- Rendle, S. Factorization machines. In *2010 IEEE International Conference on Data Mining*, pp. 995–1000. ieeexplore.ieee.org, December 2010.
- Sharma, S. Feature fusion for the uninitiated | by siddharth sharma | medium. <https://siddharth-1729-65206.medium.com/feature-fusion-for-the-uninitiated-4c5938d28a7>, 2023. (Accessed on 01/24/2024).
- Shin, K., Kwak, H., Kim, S. Y., Ramström, M. N., Jeong, J., Ha, J.-W., and Kim, K.-M. Scaling law for recommendation models: Towards general-purpose user representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 4596–4604, 2023.
- Song, W., Shi, C., Xiao, Z., Duan, Z., Xu, Y., Zhang, M., and Tang, J. AutoInt: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pp. 1161–1170, 2019.
- Tang, J., Drori, Y., Chang, D., Sathiamoorthy, M., Gilmer, J., Wei, L., Yi, X., Hong, L., and Chi, E. H. Improving training stability for multitask ranking models in recommender systems. *arXiv preprint arXiv:2302.09178*, 2023.
- Tianchi. Ad display/click data on taobao.com, 2018. URL <https://tianchi.aliyun.com/dataset/dataDetail?dataId=56>.
- Tien, J.-B., joycenv, and Chapelle, O. Display advertising challenge, 2014. URL <https://kaggle.com/competitions/criteo-display-ad-challenge>.
- Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*, pp. 1785–1797, 2021a.
- Wang, Z., She, Q., and Zhang, J. Masknet: Introducing feature-wise multiplication to ctr ranking models by instance-guided mask. *arXiv preprint arXiv:2102.07619*, 2021b.
- Zha, D., Feng, L., Luo, L., Bhushanam, B., Liu, Z., Hu, Y., Nie, J., Huang, Y., Tian, Y., Kejariwal, A., et al. Pre-train and search: Efficient embedding table sharding with pre-trained neural cost models. *Proceedings of Machine Learning and Systems*, 5, 2023.
- Zhang, G., Hou, Y., Lu, H., Chen, Y., Zhao, W. X., and Wen, J.-R. Scaling law of large sequential recommendation models. *arXiv preprint arXiv:2311.11351*, 2023.
- Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023a.
- Zhao, Z., Yang, Y., Wang, W., Liu, C., Shi, Y., Hu, W., Zhang, H., and Yang, S. Breaking the curse of quality saturation with user-centric ranking. *arXiv preprint arXiv:2305.15333*, 2023b.
- Zhu, J., Liu, J., Yang, S., Zhang, Q., and He, X. Open benchmarking for click-through rate prediction. In Demartini, G., Zuccon, G., Culpepper, J. S., Huang, Z., and Tong, H. (eds.), *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*, pp. 2759–2769. ACM, 2021. doi: 10.1145/3459637.3482486. URL <https://doi.org/10.1145/3459637.3482486>.
- Zhu, J., Dai, Q., Su, L., Ma, R., Liu, J., Cai, G., Xiao, X., and Zhang, R. BARS: towards open benchmarking for recommender systems. In Amigó, E., Castells, P., Gonzalo, J., Carterette, B., Culpepper, J. S., and Kazai, G. (eds.), *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022*, pp. 2912–2923. ACM, 2022a. doi: 10.1145/3477495.3531723. URL <https://doi.org/10.1145/3477495.3531723>.
- Zhu, J., Dai, Q., Su, L., Ma, R., Liu, J., Cai, G., Xiao, X., and Zhang, R. BARS: towards open benchmarking for recommender systems. In Amigó, E., Castells, P., Gonzalo, J., Carterette, B., Culpepper, J. S., and Kazai, G. (eds.), *SIGIR '22: The 45th International*

ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022, pp. 2912–2923. ACM, 2022b. doi: 10.1145/3477495.3531723. URL <https://doi.org/10.1145/3477495.3531723>.

A. Criteo 上针对特定模型的网格搜索空间

我们使用 Adam 优化稠密架构, 使用 Rowwise AdaGrad 优化稀疏架构, 并在前 10% 的步数中采用线性预热。我们采用 $8 * 16384 = 131,072$ 的全局批量大小。所有模型均使用 ReLU 作为激活函数。我们选择 128 作为嵌入维度, 因为在初步实验中, 它在所有模型上都表现出更好的结果。所有实验均使用 FP32。由于数据集规模和模型大小, 我们使用 (Mudigere et al., 2021) 作为稀疏分布式训练框架, 并使用数据并行进行稠密同步。

为便于公平比较, 我们在 Criteo Dataset 上对通用超参数和模型特定配置进行了广泛的网格搜索 (>3000 次运行)。

对于所有模型, 稀疏和稠密学习率分别在 $\{1e^{-3}, 1e^{-2}, 1e^{-1}\}$ 中进行调优。对于所有模型中的 MLP, 隐藏层数量取值范围为 $\{1, 2, 3, 4\}$, 层大小取值范围为 $\{512, 1024, 2048\}$ 。为了缩减过大的搜索空间, 我们对优化器超参数进行了初步实验, 发现将稠密部分的学习率设为 $1e^{-3}$ 、稀疏部分的学习率设为 $1e^{-1}$ 对所有模型效果最佳。我们在后续运行中固定了学习率。下面介绍各模型特定的搜索空间:

AFN+ 在所有运行中, AFN 隐藏单元和 DNN 隐藏单元均相同, 并遵循通用 MLP 搜索空间。对数神经元的数量取值范围为 $\{128, 256, 512, 1024\}$ 。

AutoInt+ 我们基于论文 (Song et al., 2019) 中报告的最佳配置构建搜索空间, 并为每个超参数额外考虑一个更大的值。注意力层数取值范围为 $\{3, 4\}$, 注意力维度取值范围为 $\{256, 512\}$ 。注意力头数取值为 $\{4, 8\}$ 。DNN 隐藏单元遵循通用 MLP 搜索空间。

DCNv2 交叉层数取值范围为 1 到 4。秩在满秩或 512 中搜索。

DLRM 底部 MLP 的层大小和层数设为 $[512, 256]$ 。

FinalMLP 我们遵循公开基准设置 (Zhu et al., 2022a), 将一个流的 FeatureSelection (FS) 设为所有浮点特征, 并为另一个流在 8 个选定的稀疏特征中搜索一个特征。FS MLP 设为 $[800]$ 。头数固定为 256。

MaskNet 我们同时测试了 Parallel MaskNet 和 Serial MaskNet。对于 Parallel 变体, 我们考虑 $\{1, 8, 16\}$ 中的块数量和 $\{64, 128\}$ 中的块维度。对于 Serial 变体, 我们考虑 $\{1, 4, 8\}$ 中的层数和 $\{64, 256, 1024\}$ 中的层大小。两个变体的缩减率均固定为 1。

xDeepInt 我们考虑了 Compressed Interaction Network (CIN), 其层数取值为 $\{3, 4\}$, 层维度取值为 $\{16, 32, 64\}$ 。

Wukong 底部 MLP 的层大小和层数设为 $[512, 256]$ 。 l 的取值范围为 1 到 4; n_F 和 n_L 设为相同的值, 取值范围为 $\{8, 16\}$ 。 k 固定为 24。

B. 公开数据集上的模型复杂度/大小

详情请参见表 3。

C. 模型特定的扩展配置

详情请参见表 5。

D. Wukong 中高阶交互的分析

传统因子分解机方法通过最小化 (Naumov et al., 2019) 来解决二阶交互问题:

$$\min \sum_{i,j \in S} r_{ij} - X^1 X^{1T}$$

其中, 对于 $i = 1, \dots, m$ 和 $j = 1, \dots, n$, $r_{ij} \in R$ 表示第 j 个用户对第 i 个产品的评分; X 表示用户和物品表征 (嵌入), 上标 1 表示该嵌入包含 1 阶信息。这些嵌入向量的点积能够为二阶交互产生有意义的后续评分预测。在 Wukong 中, 这些有意义的交互随后通过 MLP 转换为二阶交互表征 X^2 。在第二层 FMB 中, 通过残差连接和 LCB 连接, 点积 $(X^1 + X^2)(X^1 + X^2)^T$ 同时产生从一阶到四阶的有意义交互。依此类推, 一个具有 l 层的 Wukong 通过最小化以下目标来解决问题:

$$\min \sum_{i,j \in S} (r_{ij} - \sum_{k=1,2,\dots,2^{l-1}} X^k X^{kT})$$

因此, 与传统因子分解方法相比, Wukong 能够利用更充分的交互阶数解决推荐问题。

E. 训练数据规模的缩放定律

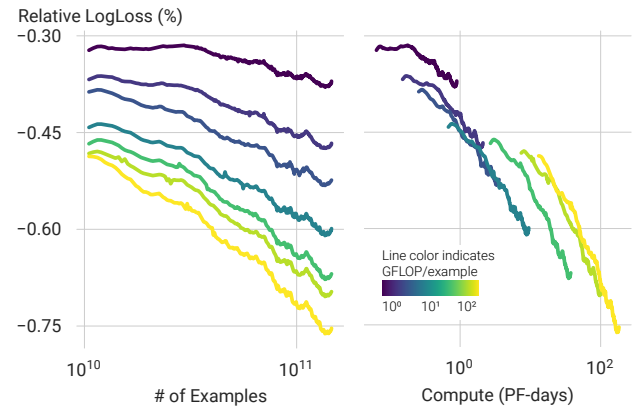


Figure 6: Wukong 的模型质量随训练数据量和训练计算量增加而提升的情况。

图 6 总结了 Wukong 的性能与其训练数据集规模 (单遍训练) 之间的关系。与在 LLM 中观察到的现象类似,

	Frappe		MicroVideo		MovieLens	
	# 参数量	MFLOP	# 参数量	MFLOP	# 参数量	MFLOP
AFN+	13607961	81.45	3205771	10.57	11259899	33.36
AutoInt+	306151	15.98	1989378	10.74	1293590	3.00
DCNv2	7251073	39.35	3454209	11.84	1243101	2.02
DLRM	1527073	4.99	2853761	12.66	1238421	1.99
FinalMLP	3115954	10.66	3020498	9.25	1040902	0.81
MaskNet	21189593	122.92	2053249	7.87	2624571	10.30
xDeepFM	66206	0.01	5147446	5.91	994439	0.00
Wukong	7769589	44.09	2219409	9.08	17734369	37.49
	KuaiVideo		TaobaoAds		Criteo	
	# 参数量	MFLOP	# 参数量	MFLOP	# 参数量	MFLOP
AFN+	84013143	10.95	167888773	13.45	26436527675	1826.50
AutoInt+	43937794	79.18	167395330	28.41	26142619137	163.34
DCNv2	42636609	9.10	193472513	166.19	26179364097	262.35
DLRM	41446513	1.99	42192033	4.53	26136307201	4.62
FinalMLP	580227666	12.17	85624114	16.39	26149500924	84.95
MaskNet	41833034	4.29	42353201	5.44	26160209153	147.40
xDeepFM	53912381	6.81	168867028	2.98	26300221171	41.57
Wukong	44671649	22.27	175724173	63.00	26163636001	173.73

Table 3: 公开数据集上的模型复杂度与规模。

我们发现大模型的数据效率更高，即它们只需更少的样本即可实现同等幅度的质量提升。此外，我们发现，所有 Wukong 模型的模型质量都持续提升至 146B 数据训练结束，而较大的模型呈现出更陡峭的模型质量提升趋势。我们还注意到，本研究的局限之一是数据集规模仍远不足以使大模型收敛，这将是未来进一步研究的方向之一。

F. 与基于 Transformer 的方法比较

我们重点说明其中的差异，并解释为何 Wukong 比 AutoInt+ (Song et al., 2019) 等基于 Transformer 的方法具有更好的扩展性。尽管 Wukong 的结构与 Transformer 相似，但我们注意到以下架构差异：首先，Wukong 在 FMB 和各层中使用的投影都是 MLP（按位），而不是 Transformer 中的 FFN（按嵌入/位置）；其次，Wukong 采用金字塔形配置，而 Transformer 使用均匀形配置。

我们推测，投影方式的差异对模型质量起着重要作用。这些 MLP 在展平后的输入嵌入上运行，实质上为每个特征提供不同的投影矩阵。我们的直觉是，这有助于模型从异构输入特征中学习，而 LLM 使用的是单一嵌入空间。(Gui et al., 2023) 中讨论了类似的直觉。

在效率方面，我们认为金字塔形配置通过逐层收缩所使用的嵌入数量，使 Wukong 能够省去不必要的计算。

为验证这些假设，我们将 Wukong 的独有组件应用于 AutoInt+ 并开展了以下实验，结果表明：(1) 在 V 投影中使用按位 MLP 代替 FFN，可使 LogLoss 改善 0.34%；(2) 在自注意力之后加入按位 MLP，可使 LogLoss 改善 0.65%；(3) 将二者结合，并采用金字塔形层结构（通过对第一层输出使用 LCB），可实现 0.57% 的质量提

升。与扩大规模的 AutoInt+ 相比，Wukong 在节省 90% FLOPs 的同时实现了 0.08% 的质量提升。我们在表 4 中汇总了结果。

变更	相对	GFLOP
相比 AutoInt+	LogLoss (%)	/样本
原始 AutoInt+	0	8
V=FFN() → V=MLP()	-0.34	13
扩大规模的 AutoInt+	-0.49	50
V=FFN() → V=MLP() 层 FFN → 层 MLP 金字塔式层形状	-0.57	5
层 FFN → 层 MLP	-0.65	36

Table 4: 将 Wukong 的独有组件替换或添加到原始 AutoInt+ 中，可提升模型质量。

超参数	GFLOP/example	#Params (B)	Relative LogLoss (任务 1)	Relative LogLoss (任务 2)
AFN+				
DNN=4x2048, afn=4x2048, nlog=1024	4.41	628.22	0.11	0.05
DNN=4x4096, afn=4x2048, nlog=1024	7.65	628.74	0.12	0.06
DNN=4x4096, afn=4x4096, nlog=2048	13.08	629.46	0.21	0.14
DNN=4x8192, afn=4x8192, nlog=4096	43.4	633.95	0.12	0.06
AutoInt+				
Attention=3x256, nhead=4, DNN=2x256	7.72	627.73	0.39	0.24
Attention=3x512, nhead=4, DNN=2x256	18.58	627.77	0.15	0.05
Attention=3x512, nhead=8, DNN=3x8192	42.53	631.49	-0.09	-0.16
Attention=3x512, nhead=16, DNN=3x10240	49.58	632.59	-0.1	-0.2
Attention=3x512, nhead=16, DNN=3x16384	68.83	635.57	0.13 (LossX)	0.01 (LossX)
DCN				
l=2, rank=512, MLP=4x2048	3	628.11	-0.27	-0.27
l=2, rank=512, MLP=4x4096	4.67	628.37	-0.29	-0.32
l=2, rank=512, MLP=4x16384	17.85	630.42	-0.38	-0.41
l=2, rank=512, MLP=4x32768	43.88	634.46	-0.43	-0.45
l=2, rank=512, MLP=4x51200	84.71	640.79	(LossX)	(LossX)
DLRM				
TopMLP=2x512	1.37	627.78	(基线)	(基线)
TopMLP=4x512	1.37	627.78	-0.11	-0.08
TopMLP=4x2048	3.85	628.17	-0.23	-0.21
TopMLP=4x4096	7.29	628.7	-0.28	-0.27
TopMLP=4x8192	14.61	629.84	-0.32	-0.31
TopMLP=4x16384	31	632.39	-0.37	-0.35
TopMLP=4x32768	71.23	638.62	-0.36	-0.34
FinalMLP				
MLP1=4x4096, MLP2=2x1024, output_dim=64, no_fs	3.93	628.25	-0.11	-0.16
MLP1=4x4096, MLP2=2x1024, output_dim=64, fs1=[0,57600], fs2=[57600,115200], fs_MLP=1x2048	8.17	628.91	-0.23	-0.27
MLP1=4x8192, MLP2=2x2048, output_dim=64, fs1=[0,57600], fs2=[57600,115200], fs_MLP=1x4096	16.9	630.27	-0.34	-0.36
MLP1=8x8192, MLP2=4x2048, output_dim=64, fs1=[0,57600], fs2=[57600,115200], fs_MLP=2x4096	18.77	630.56	-0.37	-0.38
MLP1=4x16384, MLP2=2x4096, output_dim=64, fs1=[0,57600], fs2=[57600,115200], fs_MLP=1x8192	36.26	633.27	-0.34	-0.34
MLP1=4x24576, MLP2=2x6144, output_dim=64, fs1=[0,57600], fs2=[57600,115200], fs_MLP=1x12288	58.12	636.67	-0.37	-0.38
MaskNet				
MLP=1x512, nblock=1, dim=128, reduction=0.01	1.76	627.92	-0.09	-0.12
MLP=1x512, nblock=4, dim=128, reduction=0.01	6.8	628.7	-0.22	-0.25
MLP=3x2048, nblock=4, dim=128, reduction=0.01	6.88	628.71	-0.28	-0.3
MLP=3x2048, nblock=4, dim=128, reduction=0.05	32.36	632.67	-0.37	-0.37
MLP=3x2048, nblock=4, dim=128, reduction=0.1	64.21	637.61	-0.4	-0.4
Wukong				
l=2, nL=8, nF=8, k=24, MLP=3x2048	0.53	627.74	-0.35	-0.32
l=4, nL=32, nF=32, k=24, MLP=3x2048	1.25	627.82	-0.45	-0.43
l=8, nL=32, nF=32, k=24, MLP=3x2048	2.12	627.95	-0.53	-0.49
l=8, nL=48, nF=48, k=48, MLP=3x4096	5.6	628.46	-0.6	-0.6
l=8, nL=96, nF=96, k=96, MLP=3x8192	22.23	630.96	-0.67	-0.66
l=8, nL=96, nF=96, k=96, MLP=3x16384	61	636.99	-0.7	-0.69
l=8, nL=192, nF=192, k=192, MLP=3x16384	108	644	-0.76	-0.76

Table 5: 第 7 节各次评估运行的详细超参数、计算复杂度、模型质量和模型规模。LossX 表示训练期间损失发散。